

Hermes Agent 上下文

下面按「整条链路」说明 Hermes 在 `run_agent.py` 里对 **上下文** 做了什么、为什么这么做、以及代码里是怎么落地的（聚焦 `run_conversation` 与周边）。

1. 心智模型：两类「上下文」

可以把上下文分成两层，后面所有设计几乎都围绕这两层：

类型	含义	典型内容
持久对话轨（ <code>messages</code> ）	会写进会话库、跨轮累积的对话与工具轨迹	user / assistant / tool 消息
只在当前请求里出现的临时上下文	为了省钱、缓存、或不污染历史而「只在打 API 时拼进去」	prefill、插件注入、外部 memory prefetch、 <code>ephemeral_system_prompt</code> 等

代码里明确写了：**prefill 只注入到 API 调用，不进 `messages` 列表**，因此不会进 session DB / 轨迹日志，但每次请求仍会带上。

代码块

```
1         # Prefill messages (few-shot priming) are injected at API-call time
           only,
2         # never stored in the messages list. This keeps them ephemeral: they
           won't
3         # be saved to session DB, session logs, or batch trajectories, but
           they're
4         # automatically re-applied on every API call (including session
           continuations).
```

2. 一轮对话开场：`conversation_history` → `messages`

- 1. 拷贝历史，避免改调用方传入的列表。
- 2. 追加本轮用户消息（`user_msg`）。

3. 若网关「每条消息新建一个 `AI Agent`」，内存里的 `todo` 是空的，会从历史里 **hydrate todo**，避免状态丢失。

代码块

```
1         # Initialize conversation (copy to avoid mutating the caller's list)
2         messages = list(conversation_history) if conversation_history else []
3
4         # Hydrate todo store from conversation history (gateway creates a fresh
5         # AI Agent per message, so the in-memory store is empty -- we need to
6         # recover the todo state from the most recent todo tool response in
    history)
7         if conversation_history and not self._todo_store.has_items():
8             self._hydrate_todo_store(conversation_history)
9     ...
10        # Add user message
11        user_msg = {"role": "user", "content": user_message}
12        messages.append(user_msg)
```

为什么：持久轨必须稳定、可回放；`todo` 等工具状态若只活在内存里，新实例必须从历史「捞回来」。

3. 系统提示词（System Prompt）：大块「静态上下文」

怎么拼（`_build_system_prompt`）

顺序大致是：身份（`SOUL.md` / 默认）→ 各类工具说明 → 网关传入的 `system_message` → 内置 **memory / USER 档案** → **外部 memory provider 块** → **skills** → **仓库上下文文件**（**AGENTS.md、.cursorrules** 等）→ 时间戳 / session id / model → 环境提示 → 平台提示。

代码块

```
1     def _build_system_prompt(self, system_message: str = None) -> str:
2         """
3         Assemble the full system prompt from all layers.
4
5         Called once per session (cached on self._cached_system_prompt) and only
6         rebuilt after context compression events. This ensures the system
    prompt
7         is stable across all turns in a session, maximizing prefix cache hits.
8         """
9         # Layers (in order):
10        # 1. Agent identity – SOUL.md when available, else
    DEFAULT_AGENT_IDENTITY
```

```
11         # 2. User / gateway system prompt (if provided)
12         # 3. Persistent memory (frozen snapshot)
13         # 4. Skills guidance (if skills tools are loaded)
14         # 5. Context files (AGENTS.md, .cursorrules – SOUL.md excluded here
    when used as identity)
15         # 6. Current date & time (frozen at build time)
16         # 7. Platform-specific formatting hint
17         ...
```

为什么缓存、何时重建

- **首次**：若没有「延续会话」里存下来的 prompt，就 `_build_system_prompt`，并可写入 SQLite。
- **延续会话**（尤其网关每轮新实例）：从 session DB 读出 **上一轮的 system prompt 原文**，避免重建导致与模型已见过的前缀不一致，**打断 Anthropic 等厂商的 prefix cache**。

代码块

```
1         # — System prompt (cached per session for prefix caching) —
2         # Built once on first call, reused for all subsequent calls.
3         # Only rebuilt after context compression events (which invalidate
4         # the cache and reload memory from disk).
5         ...
6         if self._cached_system_prompt is None:
7             stored_prompt = None
8             if conversation_history and self._session_db:
9                 ...
10            if stored_prompt:
11                # Continuing session – reuse the exact system prompt from
12                # the previous turn so the Anthropic cache prefix matches.
13                self._cached_system_prompt = stored_prompt
14            else:
15                # First turn of a new session – build from scratch.
16                self._cached_system_prompt =
self._build_system_prompt(system_message)
```

为什么：系统提示词通常很长且各轮相同；稳定前缀 = 更好的 KV/prompt cache 命中 = 更省延迟和钱。压缩会话后会 `_invalidate_system_prompt` 再 `_build_system_prompt`，因为记忆摘要变了，需要新的一版「冻结快照」。

4. 放进模型请求前：谁在「用户消息」里、谁在「系统」里

每一轮准备 `api_messages` 时：

1. 外部 memory 的 prefetch（整轮工具循环里只拉一次，缓存在 `_ext_prefetch_cache`）和插件 `pre_llm_call` 返回的 `context`，都拼在 当前这条 user 消息 的副本上，不修改 `messages` 里持久保存的那份。
2. 注释写得很直白：故意不进 system，为的是 不动 Hermes 控制的稳定 system 前缀（继续照顾 prompt cache）。

代码块

```
1         # Plugin hook: pre_llm_call
2         ...
3         # Context is ALWAYS injected into the user message, never the
4         # system prompt. This preserves the prompt cache prefix – the
5         # system prompt stays identical across turns so cached tokens
6         # are reused. The system prompt is Hermes's territory; plugins
7         # contribute context alongside the user's input.
8         #
9         # All injected context is ephemeral (not persisted to session DB).
```

构建单条 API 用户消息时的注入逻辑：

代码块

```
1         for idx, msg in enumerate(messages):
2             api_msg = msg.copy()
3             ...
4             if idx == current_turn_user_idx and msg.get("role") == "user":
5                 _injections = []
6                 if _ext_prefetch_cache:
7                     _fenced =
8                     build_memory_context_block(_ext_prefetch_cache)
9                     if _fenced:
10                        _injections.append(_fenced)
11                 if _plugin_user_context:
12                    _injections.append(_plugin_user_context)
13                 if _injections:
14                    _base = api_msg.get("content", "")
15                    if isinstance(_base, str):
16                        api_msg["content"] = _base + "\n\n" +
17                        "\n\n".join(_injections)
```

3. `ephemeral_system_prompt`：拼进 本次请求 的 `effective_system`，同样不进 DB 里缓存的那份「正式」system prompt。

代码块

```

1         effective_system = active_system_prompt or ""
2         if self.ephemeral_system_prompt:
3             effective_system = (effective_system + "\n\n" +
self.ephemeral_system_prompt).strip()
4         ...
5         if effective_system:
6             api_messages = [{"role": "system", "content":
effective_system}] + api_messages

```

4. `prefill_messages`：插在 system 之后、历史之前，仍是「仅 API」。

代码块

```

1         if self.prefill_messages:
2             sys_offset = 1 if effective_system else 0
3             for idx, pfm in enumerate(self.prefill_messages):
4                 api_messages.insert(sys_offset + idx, pfm.copy())

```

5. 可选：Claude + OpenRouter 的 prompt caching；工具孤儿消息清理

`_sanitize_api_messages`；空白与 tool JSON 规范化 以利于本地推理的 KV 缓存命中。

小结：「长期身份 + 记忆摘要 + 仓库规则」尽量固定在 **cached system**；「本轮检索到的记忆、插件塞进来的东西」走 **user 侧副本**，避免破坏前缀缓存，也避免把临时检索写进持久历史。

5. 超出窗口怎么办：Context Compressor / 可插拔引擎

初始化里会选 内置 `ContextCompressor` 或 插件 `context engine`（`config.yaml` 里 `context.engine`），并配阈值、保护头尾消息条数等。还可注册 上下文引擎专用工具（如 `lcm_*`）。

进入主循环前若历史已经很长，会用 粗估 token（含 tools schema）做 **preflight 压缩**，避免直接打到 API 拿到不可重试的 4xx。

代码块

```

1         # — Preflight context compression —
2         # Before entering the main loop, check if the loaded conversation
3         # history already exceeds the model's context threshold. This handles
4         # cases where a user switches to a model with a smaller context window
5         # while having a large existing session – compress proactively rather
6         # than waiting for an API error (which might be caught as a non-
retryable
7         # 4xx and abort the request entirely).
8         if (

```

```

9         self.compression_enabled
10         and len(messages) > self.context_compressor.protect_first_n
11             + self.context_compressor.protect_last_n + 1
12     ):
13         ...
14         _preflight_tokens = estimate_request_tokens_rough(
15             messages,
16             system_prompt=active_system_prompt or "",
17             tools=self.tools or None,
18         )

```

`_compress_context` 做什么（通俗版）：

1. 压缩前尽量 **flush 记忆**（避免中间删掉的细节永远丢失）。
2. 通知外部 memory 的 `on_pre_compress`。
3. 调用 `context_compressor.compress(...)` 真正缩短对话。
4. 若有 todo 快照，再追加一条 **user** 注入当前 todo。
5. 失效并重建 **system prompt**，更新 `_cached_system_prompt`。
6. 在 SQLite 里 **结束旧 session、开新 session**（带 parent 链接），便于日志与续聊管理。
7. 重置文件去重缓存等，避免「摘要后还以为文件没变」。

代码块

```

1     def _compress_context(self, messages: list, system_message: str, *,
2         approx_tokens: int = None, task_id: str = "default", focus_topic: str = None) -
3         > tuple:
4         ...
5         self.flush_memories(messages, min_turns=0)
6
7         # Notify external memory provider before compression discards context
8         if self._memory_manager:
9             try:
10                 self._memory_manager.on_pre_compress(messages)
11             except Exception:
12                 pass
13
14         compressed = self.context_compressor.compress(messages,
15             current_tokens=approx_tokens, focus_topic=focus_topic)
16
17         todo_snapshot = self._todo_store.format_for_injection()
18         if todo_snapshot:
19             compressed.append({"role": "user", "content": todo_snapshot})
20
21         self._invalidate_system_prompt()

```

```
19         new_system_prompt = self._build_system_prompt(system_message)
20         self._cached_system_prompt = new_system_prompt
```

运行中若遇到 **413、上下文溢出、Anthropic 长上下文档位限制** 等，也会走压缩/降窗口等路径（你文件里后面一长段 error handling）。

为什么：模型窗口有限；主动压缩比请求失败更可控；压缩会丢细节，所以先 flush 记忆、再拆 session，并在 UI 上提示多次压缩可能降质量。

6. 「快满了」只提醒人，不喂给模型

当用量接近 **压缩阈值** 的比例（85% / 95%）时，通过 `_emit_context_pressure` 等方式 **只给用户 / gateway 状态，不注入 messages**，避免浪费 token 且不打乱对话语义。

（逻辑在约 10211 行附近，与类级别的 `_context_pressure_last_warned` 配合，防止网关每请求新建 agent 时重复刷屏。）

7. 串起来：一句话流程

1. **持久轨：** `conversation_history` → `messages` → 必要时 hydrate todo。
2. **静态大块：**缓存的 system prompt（或从 DB 延续），内含身份、记忆快照、AGENTS 等。
3. **临时增强：**每轮 API 前复制一份消息列表，在用户消息上叠 **prefetch 记忆 + 插件上下文**；system 上叠 **ephemeral_system_prompt**；再插入 **prefill**。
4. **窗口治理：**预检 + 运行中错误触发 **压缩**；压缩会 **重建 system、新开 session、附 todo 快照**。
5. **体验：**接近阈值只 **warn 用户**，不把警告写进模型上下文。

下面按「**它是什么** → **为什么要放进静态 system prompt** → **在代码里怎么实现**」说明每一层。顺序以 `AIAgent._build_system_prompt` 里实际拼接顺序为准（和你在笔记里写的**概念顺序**几乎一致，只有 `system_message` 在代码里位于「部分工具相关段落」之后，见说明）。

SYSTEM_PROMPT

1. 身份： `SOUL.md` 与 `DEFAULT_AGENT_IDENTITY`

是什么

- `DEFAULT_AGENT_IDENTITY`：写死在 `agent/prompt_builder.py` 里的一段英文，固定说「你是 Hermes Agent、Nous Research 出品、能写代码/用工具、要直接有用」等。
- `SOUL.md`：用户放在 **HERMES_HOME**（`~/.hermes` 等）下的可选 Markdown，用来**覆盖/自定义**这段「我是谁、如何说话、默认价值观」。

为什么要包含

给模型一个稳定的「人格与底线」起点；没有仓库级规则时，至少要有厂商默认身份，避免模型把自己当成通用 ChatGPT 或其它产品。

怎么做

`load_soul_md()` 读 `get_hermes_home() / "SOUL.md"`，内容会做**注入扫描**
`_scan_context_content`（可疑模式则整块替换为 `[BLOCKED: ...]`），再按约 2 万字符做头尾截断。若有内容，第一段就用它；否则用 `DEFAULT_AGENT_IDENTITY`。

代码块

```
1 def load_soul_md() -> Optional[str]:
2     """Load SOUL.md from HERMES_HOME and return its content, or None.
3
4     Used as the agent identity (slot #1 in the system prompt). When this
5     returns content, ``build_context_files_prompt`` should be called with
6     ``skip_soul=True`` so SOUL.md isn't injected twice.
7     """
8     ...
9     soul_path = get_hermes_home() / "SOUL.md"
```

代码块

```
1 DEFAULT_AGENT_IDENTITY = (
2     "You are Hermes Agent, an intelligent AI assistant created by Nous
3     Research. "
4     ...
5 )
```

2. 各类工具说明（你已基本理解，这里补一句「实现」）

是什么

按**当前启用的工具名**条件追加的短指引，例如 `MEMORY_GUIDANCE`、`SESSION_SEARCH_GUIDANCE`、`SKILLS_GUIDANCE`（定义在同文件常量里）。

为什么

只有工具真的加载了才告诉模型「怎么用、什么时候用」，避免对没有的工具胡说。

怎么做

`_build_system_prompt` 里: `if "memory" in self.valid_tool_names` 等分支 `prompt_parts.append(...)`。

另外还有一整块 **tool-use enforcement** (是否注入由 `config` 与模型名匹配)，以及 **Gemini / GPT** 等模型家族的额外执行纪律——本质上仍是「行为约束」，和工具可用性强相关。

3. Nous 订阅能力说明 (代码里在 tool guidance 与 enforcement 之间)

是什么

`build_nous_subscription_prompt(valid_tool_names)`：若启用了 Nous 托管工具，则生成一小段「当前 Firecrawl / TTS / 浏览器等能力状态、不要向用户索要已由订阅提供的 key」等。

为什么

减少模型误判「用户要自己配 key」，并与实际托管能力对齐。

怎么做

检查 `managed_nous_tools_enabled()` 且当前工具集与订阅相关工具有交集，再拼状态行。

4. 网关 / 调用方传入的 `system_message`

是什么

`run_conversation(..., system_message=...)` 传入的字符串：例如网关里的「用户自定义指令」、产品侧 persona、合规条款等。

为什么

在 Hermes 默认身份之上，允许**部署方或用户**覆盖/叠加业务规则。

怎么做

在 `_build_system_prompt` 里: `if system_message is not None:`
`prompt_parts.append(system_message)`。

注意：注释写明 `ephemeral_system_prompt` **不在此处**，那是每轮 API 临时拼到请求里的，不进缓存的「持久 system」快照。

5. 内置 MEMORY / USER 档案

(`MemoryStore.format_for_system_prompt`)

是什么

- **MEMORY**: 模型用 `memory` 工具写入的持久笔记（偏好、环境约定等）。
- **USER**: 用户画像文件（USER.md 一类），代码里叫 user profile。

为什么

把「跨会话仍成立的事实」固定在 system 里，模型每轮都能看到；比只靠对话历史更稳。

怎么做

不是读磁盘实时内容，而是 `load_from_disk` 时拍下的快照 `_system_prompt_snapshot`。注释写得很清楚：

中途会话里 `memory` 工具写入也不会改变这一快照，目的是 **system prompt 各轮字节级一致** → 有利于 **prefix cache**。

代码块

```
1      """
2      Return the frozen snapshot for system prompt injection.
3
4      This returns the state captured at load_from_disk() time, NOT the live
5      state. Mid-session writes do not affect this. This keeps the system
6      prompt stable across all turns, preserving the prefix cache.
7
8      Returns None if the snapshot is empty (no entries at load time).
9      """
```

渲染时用 `_render_block`，带「MEMORY / USER PROFILE」标题和用量条。

6. 外部 memory provider 块

(`MemoryManager.build_system_prompt`)

是什么

插件式记忆后端（只允许多个 provider 里最多一个「外部」）通过

`MemoryProvider.system_prompt_block()` 提供的**静态说明/摘要**，由 `MemoryManager` 汇总成一段或多段文本。

为什么

内置文件记忆不足以覆盖向量库、远端记忆等服务；统一由 manager 拼进 system，和 builtin 并存。

怎么做

代码块

```

1      def build_system_prompt(self) -> str:
2          """Collect system prompt blocks from all providers.
3          ...
4          """
5          blocks = []
6          for provider in self._providers:
7              try:
8                  block = provider.system_prompt_block()
9                  if block and block.strip():
10                     blocks.append(block)

```

(**动态检索**到的段落走的是 `prefetch_all` + `build_memory_context_block`，包在 `<memory-context>` 里进**用户消息**，那是另一类，不属于这段「静态 system」拼图。)

7. Skills (`build_skills_system_prompt`)

是什么

扫描 `~/hermes/skills/` 及配置的 `external_dirs`，生成 **技能索引**：有哪些 skill、简述、分类、平台条件等（通常还带「何时应加载 skill」的说明文字）。

为什么

让模型知道「仓库里有什么可复用流程」，复杂任务优先按 skill 执行，而不是每次从零摸索。

怎么做

仅当存在 `skills_list` / `skill_view` / `skill_manage` 等 skills 相关工具时注入。内部有 **进程内 LRU 缓存 + 磁盘** `.skills_prompt_snapshot.json`（manifest 比对），避免每次冷启动全量扫盘。

8. 仓库上下文文件： `build_context_files_prompt` (AGENTS.md、.cursorrules 等)

是什么

从**工作目录**加载的项目规则，与「用户家目录里的 SOUL」不同，偏向 **当前仓库/项目怎么开发**。

实现上的**优先级**（只取一类「项目上下文」，先到先得）：

`.hermes.md` / `HERMES.md`（向上走到 git 根）→ `AGENTS.md` → `CLAUDE.md` → `.cursorrules` + `.cursor/rules/*.mdc`。

另外若 SOUL 未作为 identity 加载，`build_context_files_prompt` 里还可再贴一份 `SOUL.md`（`skip_soul=False` 时）。

为什么

把 Cursor / Claude Code 生态里常见的「项目级指令」接到 Hermes，使 agent 遵守团队约定（分支策略、测试命令、目录结构等）。

怎么做

读文件 → `_scan_context_content` 安全扫描 → 超长则 `_truncate_content`（约 2 万字符头尾保留）。

网下常用 `TERMINAL_CWD` 指定「真正的工作区」，避免 Hermes 安装目录下的 AGENTS.md 被误读（你在 `run_agent` 里见过的注释）。

代码块

```
1 def build_context_files_prompt(cwd: Optional[str] = None, skip_soul: bool =
  False) -> str:
2     """Discover and load context files for the system prompt.
3
4     Priority (first found wins – only ONE project context type is loaded):
5     1. .hermes.md / HERMES.md (walk to git root)
6     2. AGENTS.md / agents.md (cwd only)
7     3. CLAUDE.md / claude.md (cwd only)
8     4. .cursorrules / .cursor/rules/*.mdc (cwd only)
9     ...
```

9. 时间戳 / Session ID / Model / Provider

是什么

一段「对话开始时间」的格式化字符串；可选附带 `Session ID`、`Model`、`Provider`（由 `pass_session_id`、`session_id`、`model`、`provider` 控制）。

为什么

让模型知道「相对时间」和运行配置，减少「假装知道当前时间」或答错自己模型名（再配合后面 Alibaba 等特殊 workaround）。

怎么做

`hermes_time.now()` 格式化为可读时间串并 `prompt_parts.append`。

10. 环境提示（`build_environment_hints`）

是什么

描述 **工具实际跑在哪台机器上**（与「聊天平台」无关）。当前主要是 **WSL**：说明 `/mnt/c/` 与 Windows 路径对应。

为什么

用户说「桌面上的文件」时，模型必须映射到 WSL 路径，否则会读错盘。

怎么做

代码块

```
1 def build_environment_hints() -> str:
2     """Return environment-specific guidance for the system prompt.
3
4     Detects WSL, and can be extended for Termux, Docker, etc.
5     Returns an empty string when no special environment is detected.
6     """
7     hints: list[str] = []
8     if is_wsl():
9         hints.append(WSL_ENVIRONMENT_HINT)
10    return "\n\n".join(hints)
```

11. 平台提示（PLATFORM_HINTS）

是什么

按 `self.platform`（如 `whatsapp`、`cli`、`email`、`cron`）选取的一段固定文案：是否避免 Markdown、是否用 `MEDIA:/path` 发附件、SMS 长度限制等。

为什么

同一套 Agent 核心在不同通道上 **输出形态必须不同**（邮件 plain text、Cron 不能反问用户等）。

怎么做

```
platform_key = (self.platform or "").lower(), if platform_key in PLATFORM_HINTS: prompt_parts.append(PLATFORM_HINTS[platform_key])。
```

下面把「用户消息周边」的几件事拆开说明：**各自典型内容是什么、为什么要单独一类、在代码里怎么接到模型上**。你已经理解的「复制 API 消息、持久轨不注入插件/memory」我不再重复，只补充它们和这几类的关系。

USER_PROMPT

它们解决的是**不同维度**的问题，叠在一起看起来像「很多」，但职责可以分成四块：

维度	这类机制管什么
放在消息的哪个槽位	<code>system</code> （拼接在缓存 system 后面）vs 伪造的多轮对话 （prefill）vs 当前用户气

	泡里的附录 (prefetch / pre_llm_call)
内容从哪来	用户配置 (YAML/环境变量)、网关单次请求、插件、外部记忆后端
变不变	整轮工具循环里固定 vs 每轮开始时算一次
要不要写进 session DB	这几类都不写进持久 messages，避免污染历史与破坏前缀缓存

所以不是「同一件事写了四遍」，而是：**稳定的 Hermes system 一块 + 可选的长期 persona (ephemeral system) + 可选的 few-shot 假对话 (prefill) + 按查询检索出来的临时背景 (prefetch + 插件)。**

1. prefetch (MemoryManager.prefetch_all)

是什么

在外部记忆插件 (Honcho、Mem0、Hindsight、Holographic、Supermemory…) 里，用**本轮用户问题** (代码里用 original_user_message，避免被 skill 等前缀污染) 去拉一段「和当前问题相关的检索结果」。每个 provider 的 prefetch(query) 返回一段字符串，prefetch_all 把各段用空行拼起来。

典型内容

- Mem0: 向量检索 top 几条 memory，格式化成 bullet。
- Honcho: 后台线程里算好的 dialectic / recall，可能带 ## Honcho Context 标题。
- Hindsight: reflect / recall 返回的文本。
- Holographic: 按 trust 检索的事实列表。

内置文件 memory (memory 工具那条链) 默认 prefetch() 基类实现返回空字符串——静态快照已在 _build_system_prompt 里；动态检索是插件的事。

为什么要它

静态塞进 system 的只有「加载时快照 + provider 的 system_prompt_block」，**不能**每轮按问题检索；prefetch 相当于「这次用户问了什么，就从向量库/图数据库捞什么」，而且可以在 provider 里用后台线程、queue_prefetch 做_latency 优化。

怎么做

在 `run_conversation` 里，进入工具循环前调用一次 `prefetch_all`，结果缓存在 `_ext_prefetch_cache`；每一次准备 `api_messages` 时，只在**当前这条 user**的副本后面追加，并用 `build_memory_context_block` 包成 `<memory-context>...</memory-context>`，标明「这是系统召回，不是用户新说的话」。

2. `pre_llm_call`（插件钩子）

是什么

Hermes CLI 插件系统在**每轮、进入主循环之后、第一次**（以及每次迭代准备请求时逻辑上挂在同一轮）调用的钩子：插件可以返回字符串或 `{"context": "..."}` ，收集后拼进 `_plugin_user_context`，和 `prefetch` 一样追加到当前用户消息的 API 副本上。

典型内容

完全由你装的插件决定：例如当前仓库 git 分支、IDE 里打开的文件、内部工单状态、仅限本产品的路由提示等。

为什么要它

这是 **扩展点**：不改 Hermes 核心就能注入「产品侧上下文」。刻意放在 **user 消息** 而不是 system，注释写明是为了 **不打乱稳定的 system 前缀**（prefix / KV cache）。

怎么做

`invoke_hook("pre_llm_call", ...)`，合并所有插件返回的 context；**不入库**。

3. `ephemeral_system_prompt`

是什么

一段追加在「已缓存的完整 system prompt」后面的字符串，只在构造发给模型的 `effective_system` 时出现；**不参与** `_build_system_prompt` 里那份「可持久化 / 可缓存」的拼接（注释写明不包含它）。

典型内容（常见来源）

1. **Gateway 全局配置**：环境变量 `HERMES_EPHEMERAL_SYSTEM_PROMPT`，或 `~/hermes/config.yaml` 里 `agent.system_prompt`。
2. **单次会话层**：Gateway 在 `run_sync` 里把 `context_prompt`（与当前聊天/频道相关的说明）和上面的全局 `_ephemeral_system_prompt` 拼成 `combined_ephemeral`，再传给 `AIAgent`。
3. **API 客户端**：测试里可见客户端可把「instructions」映射到 `ephemeral_system_prompt`。

4. 子代理: `delegate_tool` 会把子任务专用提示作为 `ephemeral_system_prompt` 传给子 agent。

为什么要它

- 人格 / 口吻 / 通道规则若写进「持久 system」有时会参与 session DB、压缩逻辑；这里强调 **仅执行期**、且常与「用户配置的额外指令」绑定。
- 与 **冻结的 Hermes 大块 system** 分离：缓存的主 prompt 仍是身份+记忆快照+AGENTS.md；**本轮额外规则**用 ephemeral 叠上去。

怎么做

`effective_system = active_system_prompt + ephemeral_system_prompt`（再与 ... 拼成最终第一条 system message）。**不写进** `_cached_system_prompt` 快照（因此也不会因为 ephemeral 变化而整块重建静态层）。

4. `prefill_messages`

是什么

一个 OpenAI 格式的消息列表（如 `[{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]`），在代码里插入在 **system 之后、真实对话历史之前**。看起来像多轮对话，但 **不会** append 到持久 `messages`。

典型内容

- **配置文件**： `prefill_messages_file` 指向一个 JSON 数组（CLI/Gateway 会从 `HERMES_PREFILL_MESSAGES_FILE` 或 config 读）。用于 **few-shot**（示范用户怎么问、助手怎么答）、或固定「角色演练」对话。
- **评测 / 脚本**：例如 red-team 脚本里自动生成 prefill JSON。

为什么要它

和「写在 system 里的一段文字」相比，**多轮示例**对很多模型更听话（模仿格式、工具用法、语气）。放在 **system 之后、历史之前**，模型会先读到示范再读真实会话。

怎么做

Gateway `AIAgent(..., prefill_messages=self._prefill_messages)`；
`run_conversation` 每次组 `api_messages` 时在 system 后 `insert`。Cron 调度也有同样加载逻辑。

5. 和其它细节的关系（帮你串一下）

- `original_user_message` vs `user_message`：prefetch 用前者，避免把语音前缀、skill 注入等塞进检索查询导致检索跑偏或过长。
- 整轮只 prefetch 一次：`_ext_prefetch_cache` 在 while 循环外填充，避免 10 次工具调用打 10 次向量库。
- 插件与 prefetch 都进 user 副本：同一位置拼接，顺序是先 fenced memory block，再插件 context。
- Unicode / ASCII 清理：部分路径会对 `prefill`、`ephemeral_system_prompt` 做 sanitize，避免个别 API 拒绝非 ASCII——仍是「请求副本」层面。

一句话对照表

机制	更像什么	常见内容谁决定
prefetch	按本轮问题的「临时参考资料」	各 MemoryProvider 的检索实现
pre_llm_call	产品/插件定制的「侧边栏信息」	用户安装的插件
ephemeral_system_prompt	不改变主缓存的前提下多叠一层「系统规矩」	config、网关 context、API instructions、子代理
prefill_messages	插在开头的「假对话示范」	JSON 文件 / 配置

如果你关心「Gateway 里 `context_prompt` 具体会包含哪些字段」，那是在 `gateway/run.py` 里拼 `combined_ephemeral` 之前的分支逻辑，需要的话可以指定你从哪条通路（Telegram / API / CLI）进来，我可以按那条调用链再往下指到函数名。

下面只讲「上下文压缩」这一条链：配置怎么定阈值、`ContextCompressor` 里具体怎么做、`run_agent.py` 在什么时机调用、以及和「报错再压」相关的分支。

Context Handling

1. 阈值和预算：先定「多大算危险」，再定「尾巴留多长」

压缩器在 `AI Agent` 初始化时挂上，配置来自 `config.yaml` 的 `agent.compression`（以及可选的按模型覆盖阈值）。核心参数是：

- `enabled`：关则整条自动压缩链（含 preflight / 工具轮后检测）都不走「主动压」逻辑（错误路径里是否还压要看具体分支，但通常用户会开着）。
- `threshold`：默认 `0.50`，表示 **主模型上下文长度 × 该比例 = `threshold_tokens`**，超过就认为该考虑压缩了。还有一个下限 `MINIMUM_CONTEXT_LENGTH`，避免超大窗口时 50% 仍高得离谱或反过来过小——见 `ContextCompressor.__init__` / `update_model`。

代码块

```
1         self.threshold_tokens = max(
2             int(self.context_length * threshold_percent),
3             MINIMUM_CONTEXT_LENGTH,
4         )
5         ...
6         target_tokens = int(self.threshold_tokens * self.summary_target_ratio)
7         self.tail_token_budget = target_tokens
```

- `target_ratio`（代码里叫 `summary_target_ratio`，默认 0.20）：**不是**「压到总上下文的 20%」，而是 `threshold_tokens` 的 20% 用来推导 `tail_token_budget`——即「尾部保护区」大致允许多少 token 的近期对话**尽量完整保留**，中间再拿去摘要。
- `protect_last_n`：在「按 token 算尾巴」时作为 **最少条数地板**（和 `tail_token_budget` 一起用，token 优先、条数保底）
- **辅助模型 `auxiliary.compression`**：摘要阶段走 `call_llm` 那套「辅助任务」解析链（便宜/快的模型）。`run_agent` 里 `_check_compression_model_feasibility` 会检查 **压缩模型窗口是否装得下「要送进摘要器的那坨 transcript」**，否则降阈值或告警——避免摘要调用本身 OOM。

2. 内置压缩算法（`ContextCompressor.compress`）在干什么

入口注释把算法写成了五步，实现上可以再拆细一点理解：

代码块

```
1     def compress(self, messages: List[Dict[str, Any]], current_tokens: int =
2         None, focus_topic: str = None) -> List[Dict[str, Any]]:
3
4         """Compress conversation messages by summarizing middle turns.
5
6         Algorithm:
7         1. Prune old tool results (cheap pre-pass, no LLM call)
8         2. Protect head messages (system prompt + first exchange)
```

- | | |
|---|---|
| 7 | 3. Find tail boundary by token budget (~20K tokens of recent context) |
| 8 | 4. Summarize middle turns with structured LLM prompt |
| 9 | 5. On re-compression, iteratively update the previous summary |

(1) 廉价预处理：修剪旧 tool 输出

`_prune_old_tool_results`：在不动「尾巴」的前提下，把 **较早的 tool 结果** 换成一行摘要或占位、去重相同结果、截断尾巴外 assistant 里过长的 `function.arguments` JSON。这样 **有时只靠这一步就能明显降 token**，不必每次都调用摘要 LLM。

(2) 头部保护区

默认 `protect_first_n = 3`（构造参数），并且用 `_align_boundary_forward` 保证 **不会从半截 tool 链开始切**——否则后面 `_sanitize_tool_pairs` 会丢配对，模型看到的历史是坏的。

(3) 尾部：按 token 预算，而不是死板「最后 20 条」

`_find_tail_cut_by_tokens`：从末尾往前累加估算 token，预算来自上面的 `tail_token_budget`，另有 **至少 3 条消息、软上限 $1.5 \times \text{预算}$** 等细节，避免一刀砍在巨型单条 tool 输出中间。还有 `_ensure_last_user_message_in_tail`：防止边界对齐后把「最后一条用户任务」误送进中间摘要区，导致模型认为「没有最新用户指令」（修了 #10896）。

(4) 中间区：LLM 结构化摘要

`compress_start` 到 `compress_end` 之间的消息交给 `_generate_summary`（内部用辅助模型；可选 `focus_topic` 做 `/compress` 那种定向压缩）。若已有旧摘要块，会 **迭代更新** `_previous_summary`，而不是每次从零写，减轻多轮压缩后信息漂移。

(5) 拼回消息列表

- 头部消息原样拷贝；若第一条是 `system`，会附加一段 **说明发生过 compaction** 的 note。
- 中间变成 **一条带 `SUMMARY_PREFIX` 等前缀的 summary 消息**（角色 `user` / `assistant` 会算一遍，避免和头尾 **连续同角色**；实在不行就把摘要 **prepend 进尾巴第一条**）。
- 尾部消息原样接上。
- 最后 `_sanitize_tool_pairs` 清理孤儿 tool，保证发给 API 的历史合法。
- 更新 `compression_count`，并根据压缩前后粗估 token 算 `savings_pct`，用于下面的「防抖动」。

摘要失败时：插入静态 fallback 文本，并记录 `_last_summary_error` / `_last_summary_dropped_count` 等，`run_agent._compress_context` 里会向用户发警告（避免「静默丢了几百条」）。

3. 「该不该压」：`should_compress` 的防抖动

工具循环里用的是 `should_compress(prompt_tokens)`，不是简单 `tokens > threshold`：

代码块

```
1 def should_compress(self, prompt_tokens: int = None) -> bool:
2     ...
3     tokens = prompt_tokens if prompt_tokens is not None else
    self.last_prompt_tokens
4     if tokens < self.threshold_tokens:
5         return False
6     # Anti-thrashing: back off if recent compressions were ineffective
7     if self._ineffective_compression_count >= 2:
8         ...
9         return False
10    return True
```

每次 `compress()` 结束会根据 `savings_pct < 10%` 累加 `_ineffective_compression_count`，否则清零。含义是：**如果连续两次压缩几乎没省下空间（例如中间已没什么可压、全是巨型尾巴 tool），再压只会白烧摘要模型钱/时间，还可能扰动对话，所以直接跳过**，日志里提示用户考虑 `/new` 或带主题的 `/compress`。

4. `run_agent.py` 里「什么时候压」——三条主路径 + 若干错误分支

路径 A: Preflight（进主循环之前）

在拼好 `messages`（含本轮用户句）之后、进入 `while` 工具循环之前：

- 条件：`compression_enabled` 且消息条数足够多（超过 `protect_first_n + protect_last_n + 1` 之类 guard）。
- 用 `estimate_request_tokens_rough(messages, system_prompt=..., tools=...)` 估 token——刻意带上 `tools`，避免低估 50+ 工具 schema 那几万 token
- 若 `#tokens >= threshold_tokens`：发状态、最多 3 轮 pass 调 `_compress_context`；每轮后若新会话则 `conversation_history = None`（避免 DB flush 跳过已压缩段）；并重置若干 empty-retry 计数，避免压缩后立刻被旧重试状态坑死。

目的：换小窗口模型、或 resume 超长会话时，**别等第一次 API 就 400/413**，先压到阈值以下再说。

路径 B: 工具循环内「健康检查式」压缩（你说的「撑爆前主动压」的主轴）

每次带 `tool_calls` 的 `assistant` 回合处理完（tool 结果都写回 `messages` 后），在继续下一轮 API 之前：

代码块

```
1          # Use real token counts from the API response to decide
2          # compression. prompt_tokens + completion_tokens is the
3          # actual context size the provider reported plus the
4          # assistant turn – a tight lower bound for the next prompt.
5          ...
6          if _compressor.last_prompt_tokens > 0:
7              # Only use prompt_tokens – completion/reasoning
8              # tokens don't consume context window space.
9              ...
10             _real_tokens = _compressor.last_prompt_tokens
11         else:
12             ...
13             _real_tokens = estimate_request_tokens_rough(
14                 messages, tools=self.tools or None
15             )
16
17         if self.compression_enabled and
18         _compressor.should_compress(_real_tokens):
19             self._safe_print(" ⚡ compacting context...")
20             messages, active_system_prompt =
21             self._compress_context(
22                 ...
23             )
24             conversation_history = None
```

要点：

- `last_prompt_tokens` 来自上一次成功 API 响应的 `usage.prompt_tokens` (`update_from_response`)，比纯本地估算准。
- 刻意 **不用 completion 里 reasoning inflated 的部分** 当「窗口压力」(#12026)，否则思考模型会 **过早狂压**。
- 若 usage 丢失 (0)，回退到 **带 tools 的 rough estimate**，避免压缩永远不触发 (#2153)。

路径 C：报错/限流后的「救命压」

在同一次 API 调用的 `retry_count` 内层循环里，错误分类器会标出 `should_compress`、`payload_too_large`、`context_overflow`、`long_context_tier` 等。你关心的包括：

1. **HTTP 413**：请求体过大 → 增加 `compression_attempts`，在 ≤3 次内调用 `_compress_context`，`restart_with_compressed_messages = True` 跳出重试、用新历史再打 API。
2. `FailoverReason.payload_too_large`（非 HTTP 413 的同类语义）：同样走压缩重试，超过 3 次则 `compression_exhausted` 返回。

3. `context_overflow` / 上下文相关 400：会先尝试 `get_next_probe_tier` 下调 `context_length` 或从错误信息里 解析真实上限，`update_model` 后再压；仍过大则再 `_compress_context`。
4. Anthropic 「长上下文档位」 429：把 `context_length` 降到 200000 再压（订阅档限制，不是模型真能力），再压缩重试。

这些分支共享一个思路：先让「认为的窗口」和提供商一致，再通过压缩把 transcript 砍进安全区，且 `compression_attempts` 上限为 3，防止死循环。

5. `_compress_context`：不只是「改 messages」

你在上一问里已经见过 `_compress_context`；和「压缩算法」衔接时值得强调：它是编排层，在 `compress()` 返回之后还会：

- `on_pre_compress` / `commit_memory_session` / 换 `session_id`、SQLite 会话分裂、`_invalidate_system_prompt` + 重建、todo 注入、`reset_file_dedup`、更新 `last_prompt_tokens` 为压缩后全请求粗估值等。

这样 算法层（ContextCompressor）只负责「消息列表怎么变」，产品层（run_agent）负责会话边界、记忆、计费相关状态。

6. 和插件式 Context Engine 的关系

`run_agent` 初始化里若 `agent.context.engine` 配了非 `compressor` 的名字，会选用插件 `ContextEngine`；`compress()` 签名不兼容时会 去掉 `focus_topic` 再调。行为细节以插件为准，但 `run_agent` 的触发时机（`preflight` / `should_compress` / 413 / `overflow`）大体不变。

小结表

阶段	依据	做什么
Preflight	粗估全请求 token（含 system + tools）≥ 阈值	可能多轮 <code>_compress_context</code>
每轮工具后	上次 API 的 <code>prompt_tokens</code> （或回退估算）+ <code>should_compress</code>	调用 <code>compress()</code> ，带防抖动
413 / overflow / tier	错误分类 + <code>compression_attempts</code>	降探测窗口或压历史再重试

	≤ 3	
compress() 内部	消息结构与 token 预算	先剪 tool → 定头尾界 → LLM 摘要 → 修 tool 对

流程图

```
流程图

1  Hermes 上下文压缩 (文本流程图)
2  =====
3
4  构造/配置 AIAgent
5  |
6  |- 加载 config: compression.enabled / threshold / target_ratio / protect_last_n
7  |- 实例化 context_compressor (内置 ContextCompressor 或插件 ContextEngine)
8  |- 计算 threshold_tokens、tail_token_budget; compression_attempts 本轮初值 = 0
9  |
10 v
11 run_conversation(user_message, conversation_history, ...)
12   (CLI / Gateway 等入口)
13 |
14 |- 拷贝 messages <- history; append 本轮 user
15 |- 准备 _cached_system_prompt (新会话 build / 续会话读 DB)
16 |
17 |-+--- 回合前: Preflight 压缩 -----
18 | |
19 | | |- compression_enabled ?
20 | |   否 -> 跳过
21 | |   是 -> 条数 > protect_first_n + protect_last_n + 1 ?
22 | |       否 -> 跳过
23 | |       是 -> rough = estimate_request_tokens_rough(messages,
    | |               system, tools)
24 | |               rough >= threshold_tokens ?
25 | |               否 -> 跳过
26 | |               是 -> 状态「Preflight 压缩...」
27 | |                   for pass in 1..3:
28 | |                       _compress_context(...)
29 | |                       conversation_history = None    (新 session 时)
30 | |                       重置部分 retry 计数
31 | |                       重算 rough; 若 < threshold -> break
32 | |
33 | v
34 |
35 |-+--- while 主循环 (iteration_budget + max_iterations) -----
36 | |
37 | |- 中断 / 预算耗尽 ? -> break
```

```

38 | |
39 | |- 组 api_messages (拷贝 messages; user 条注入 prefetch / 插件上下文; 拼
    system)
40 | |- sanitize / 规范化 (api_messages 副本)
41 | |
42 | |--- while 内层「单次 API 重试」 (retry_count < max_retries) --
43 | | |
44 | | | |- _build_api_kwargs -> streaming 或普通 API 调用
45 | | |
46 | | | |- 成功 ?
47 | | | 是 -> update_from_response(usage) // last_prompt_tokens =
    prompt_tokens
48 | | | compression_attempts = 0 (部分成功路径会清零)
49 | | | 跳出内层 retry
50 | | |
51 | | L- 失败 -> 错误分类 (413 / payload_too_large / context_overflow /
    long_context_tier ...)
52 | | |
53 | | | |- 413 或 payload_too_large ?
54 | | | | compression_attempts++
55 | | | | attempts <= 3 ?
56 | | | | 是 -> _compress_context(...); conversation_history=None
57 | | | | restart_with_compressed_messages -> break 内层, 重开大循
    环迭代
58 | | | L- 否 -> 返回 compression_exhausted / 错误
59 | | |
60 | | | |- context_overflow / 可压的 400 ?
61 | | | | 可能 update_model (probe 降 context_length / 解析上限)
62 | | | | compression_attempts++
63 | | | | attempts <= 3 ?
64 | | | | 是 -> _compress_context(...)
65 | | | L- 否 -> ... 其它分支 (fallback / 非重试错误等)
66 | | |
67 | | | L- long_context_tier (Anthropic 1M 档) ?
68 | | | context_length 降到 200k + _compress_context (同 attempts 逻辑)
69 | | |
70 | | L---- (内层 retry 结束) -----
71 | | |
72 | | |- 本轮 assistant 有 tool_calls ?
73 | | 是 -> 写回 assistant + 执行 tools -> 写 tool 结果到 messages
74 | | |
75 | | | |- real_tokens = last_prompt_tokens > 0 ? last_prompt_tokens
76 | | | | : estimate_request_tokens_rough(messages,
    tools)
77 | | |
78 | | | |- compression_enabled && should_compress(real_tokens) ?

```



```

79 | | | (should_compress: >= threshold 且 非「连续两次节省<10%」防
    抖)
80 | | | 是 -> _compress_context(...); conversation_history=None
81 | | | L- 否 -> 不压
82 | | | |
83 | | | L- continue 主循环
84 | | |
85 | | L- 否 (无 tool_calls) -> 定稿 final_response -> break 主循环
86 | |
87 | | - 回合后: persist / memory sync / ...
88 | |
89 v
90 结束
91
92
93 _compress_context (编排层) ContextCompressor.compress (算法
    层)
94 =====
    =====
95
96 _compress_context(messages, system_message) compress(messages,
    current_tokens, focus_topic?)
97 | |
98 | - on_pre_compress (外部 memory 插件) L- 消息条数 <= protect_first_n +
    3 + 1 ?
99 | - context_compressor.compress(...) 是 -> 原样返回 messages
100 | |
101 | - 摘要失败? -> 警告 / fallback 文本 L- Phase1
    _prune_old_tool_results (剪旧 tool)
102 | - todo_snapshot 有? -> append 伪 user 条 |
103 | - _invalidate_system_prompt; 重建 system L- Phase2 定界
104 | - SQLite: end_session + 新 session_id compress_start =
    align(protect_first_n)
105 | - commit_memory / on_session_switch ... compress_end =
    _find_tail_cut_by_tokens
106 | - reset_file_dedup (保证最后 user 在 tail, 不
    切断 tool组)
107 | - last_prompt_tokens <- 压缩后全请求粗估 |
108 | | L- Phase3 _generate_summary(中间
    段) // 辅助LLM
109 L- return (compressed_messages, new_system) |
110 | L- Phase4 拼装: head +
    summary_msg + tail
111 | L- _sanitize_tool_pairs
112 | L- compression_count++
113 | L- 更新 ineffective 计数 (节省
    <10% 累加)

```

一些问题

问题1：在上下文被压缩之后，我记得好像会重建system_message？为什么需要重建呢？重建后的system_message和之前的主要区别是什么？在压缩上下文的时候，不是会保留前部和尾部的部分信息嘛？

1、为什么要重建

压缩只处理 `messages` 里的对话条（头尾保留、中间变摘要）。`system` 在 Hermes 里是 `_cached_system_prompt` 单独维护的，不跟「消息列表前几条」绑在一起。压缩后会 **换新 session_id**、SQLite 新开一行会话，volatile 里那几行（时间、Session ID 等）必须变；同时 `_invalidate_system_prompt` 会 `load_from_disk()`，把本轮里可能已改过的 **MEMORY/USER** 再读进 system。不重建的话，模型看到的是 **旧 session 元数据 + 可能过期的 memory 快照**，和真实会话状态不一致。

2、和压缩前比，主要差在哪

- **变的多**：volatile 一段（memory/user 块、外部 memory 的 system 块、带 **新 session_id** 的时间戳行等）。
- **大多不变**：stable（身份、工具说明、技能说明等）和 context（`AGENTS.md` 等）在文件/配置没变时通常和之前差不多。

3、「不是保留了前部尾部吗」

保留的是 **对话 transcript 的头尾消息**，不是「system 字符串自动等于头部」。所以 **摘要负责丢中间对话**；**重建 system 负责会话边界 + 持久记忆快照与元信息对齐**。两件事互补。